

Repo Spine

GITHUB Repo-

[model-engineering/README.md at main · ashishnitk/model-engineering](#)

The **repo spine** is the **core project structure** that stays the same throughout the course. It is called a “spine” because everything important attaches to it: data, preprocessing, models, evaluation, experiments, and reports. Instead of creating messy notebooks every week, Students keep building on one organised repository.

What the Repo Spine Does

It gives you a real ML workflow:

- Data is stored in a predictable place
- Reusable code lives in src/
- Experiments are logged consistently
- Configs control behaviour
- Reports document decisions
- Final artefacts are easy to trace and reproduce

So the repo spine is not just folder organisation. It is what makes the project **repeatable, auditable, and scalable**.

Example Repo Sine

```
repo/
├── data/
│   ├── raw/
│   ├── processed/
│   └── data_dictionary.md
├── src/
│   ├── features/
│   ├── models/
│   ├── eval/
│   ├── inference/
│   └── utils/
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_feature_experiments.ipynb
│   └── ...
├── configs/
│   ├── data/
│   ├── features/
│   ├── models/
│   └── train/
├── runs/
└── run_001/
```

```
|— run_002/  
|— summary.csv  
|— reports/  
|   |— weekly/  
|   |— evaluation/  
|   |— model_card_v1.md  
|— train.py  
|— evaluate.py  
|— predict.py  
|— smoke_test.py  
|— requirements.txt  
|— README.md
```

Folder-by-Folder Explanation

data/

This stores the dataset and related documentation.

data/raw/

Original data as received.

Purpose:

- preserve the source data
- Avoid overwriting original files
- Support reproducibility

Students should treat raw data as read-only.

data/processed/

Cleaned or transformed data ready for modelling.

Purpose:

- save prepared datasets
- separate preprocessing outputs from raw input
- make reruns cleaner

data_dictionary.md

A human-readable description of columns.

Should include:

- column name
- type
- meaning
- possible values
- warnings about leakage or missingness

This helps students and graders understand the dataset without guessing.

src/

This contains the real reusable code.

This is where final logic should live after students prototype in notebooks.

src/features/

Code for preprocessing and feature engineering.

Examples:

- missing value handling
- encoding
- scaling
- transformations
- feature creation
- leakage-safe preprocessing pipeline

Why it matters:

Feature logic should not stay trapped in notebooks.

src/models/

Code for model training and model definitions.

Examples:

- baseline model code
- training utilities
- model selection logic
- model saving/loading

src/eval/

Evaluation code.

Examples:

- metric computation
- threshold sweep
- ROC/PR plots
- clustering evaluation
- calibration analysis
- slice metrics

Why it matters:

evaluation should be consistent across runs.

src/inference/

Code used to make predictions from a trained artifact.

Examples:

- input schema checks
- preprocessing + prediction pipeline
- batch or single-row prediction helpers

This makes the project feel closer to deployment.

src/utils/

Shared helper functions.

Examples:

- logging helpers
- config loading
- seed setting
- path handling

This avoids repeating the same code everywhere.

notebooks/

This is for exploration and experimentation.



Typical notebooks:

- EDA
- trying a transformation
- comparing feature ideas
- plotting results

Important rule:

use notebooks for thinking, but move final reusable logic into src/.

That is a key course habit.

configs/

This stores settings that control how the pipeline runs.

Instead of hardcoding everything in Python, students keep settings in YAML or JSON.

configs/data/

Things like:

- dataset path
- target column
- split method
- time cutoff

configs/features/

Things like:

- selected columns
- feature set ID
- encoding choices
- scaling strategy

configs/models/

Things like:

- model type
- hyperparameters
- regularisation settings

configs/train/

Things like:

- seed
- batch size
- epochs
- CV settings
- output paths

Why configs matter:

They make experiments easier to compare and rerun.

runs/

This stores experiment outputs.

Each run can have its own folder with:

- params
- metrics



- plots
- trained model artefact
- feature config used
- dataset version used

Example:

runs/run_001/

```
|— params.json
|— metrics.json
|— model.pkl
|— feature_config.json
|— notes.txt
```

Purpose:

- compare experiments cleanly
- preserve training history
- support traceability

This is where students prove what they ran.

reports/

This is for human-readable project communication.

reports/weekly/

Short weekly summaries.

Examples:

- What changed this week
- best run so far
- key risks
- next steps

reports/evaluation/

Formal comparison reports.

Examples:

- model bakeoff report
- threshold analysis
- clustering brief
- error analysis

model_card_v1.md

Final model documentation.

Usually includes:

- intended use
- training data summary
- features used
- metrics
- failure modes
- threshold choice
- monitoring/drift risks

This teaches students to communicate model limitations, not just scores.



Top-level Scripts

train.py

Runs model training end-to-end.

Purpose:

- one command to reproduce training
- load config
- set seed
- load data
- preprocess
- train model
- save artefact
- log run

This becomes very important by Week 14.

evaluate.py

Runs evaluation on saved predictions or trained models.

Purpose:

- compute metrics consistently
- generate plots and reports

predict.py

Inference entrypoint.

Purpose:

- load artefact
- run preprocessing
- make predictions on new data

smoke_test.py

Quick test to confirm the pipeline works.

Purpose:

- catch broken paths, schemas, or serialisation problems
- run a tiny sample through the full inference flow

This is like a sanity check before submission or deployment.

requirements.txt

Lists required Python packages.

Purpose:

- environment reproducibility
- easier setup for graders or teammates

README.md

The project's front door.

Should explain:

- What the project is
- How to install dependencies
- How to run training
- how to run evaluation



- where outputs are saved

A good README makes the repo usable by someone else.

Why does This Matter in the Course

The repo spine prevents students from doing this:

- storing final logic only in notebooks
- manually renaming files like final_v2_reallyfinal.ipynb
- losing track of which model used which features
- forgetting how a result was produced
- mixing raw and processed data
- overwriting important outputs

Instead, it builds habits used in real ML projects:

- structure
- reproducibility
- separation of concerns
- traceability

Simple Mental Model

You can explain the repo spine like this:

- data/ = what we work on
- src/ = how we work on it
- notebooks/ = where we explore ideas
- configs/ = what settings we choose
- runs/ = what happened in each experiment
- reports/ = what we learned
- top-level scripts = how to rerun the whole workflow

In One Sentence

The **repo spine** is the stable project structure that lets students build, test, compare, document, and reproduce one ML system over multiple weeks without losing organisation.